

Algorithms

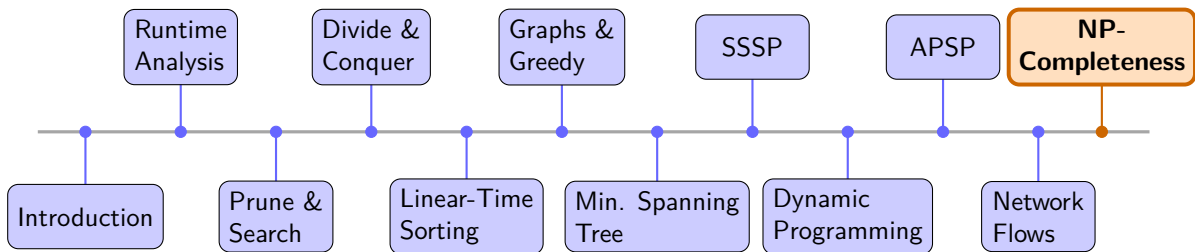
Dr. Mudassir Shabbir

LUMS University

April 22, 2026



Recap & What's Next



NP-Completeness

Lecture 3: SAT, 3SAT, 3COL, VC and Key Reductions



Recall: Class P

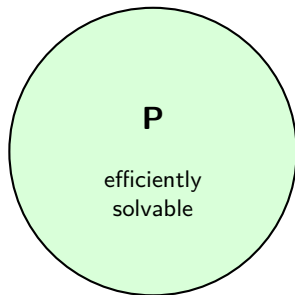
Definition

P is the class of decision problems solvable in **polynomial time**.

This means there is an algorithm with running time $O(n^k)$ for some constant k .

Examples in P:

- Shortest path
- Minimum spanning tree
- Bipartite testing
- Sorting



Recall: Class NP

Definition

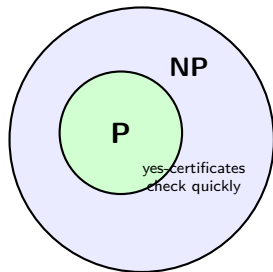
NP is the class of decision problems whose **Yes**-instances have certificates verifiable in **polynomial time**.

Think: a solution may be hard to find, but easy to check.

Examples in NP:

- SAT: certificate is a truth assignment
- VC: certificate is the chosen vertex set
- 3COL: certificate is the coloring

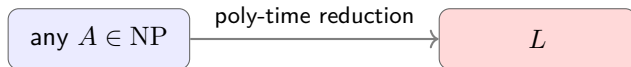
$P \subseteq NP$ because solving is at least as strong as verifying.



Recall: NP-Hard

Definition

A problem L is **NP-hard** if every problem in NP can be reduced to L in polynomial time.



Interpretation: if we could solve L efficiently, then we could solve every problem in NP efficiently.

Important

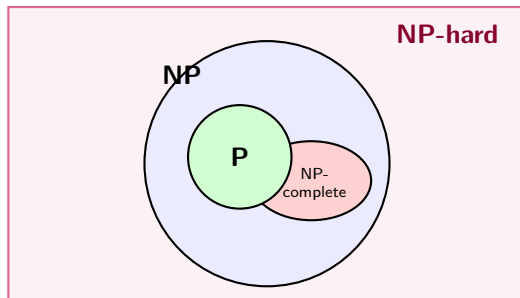
NP-hard does *not* mean the problem is in NP. It only says the problem is at least that hard.

New Definition: NP-Complete

Definition

A decision problem is **NP-complete** if:

- 1 it is in NP, and
- 2 it is NP-hard.



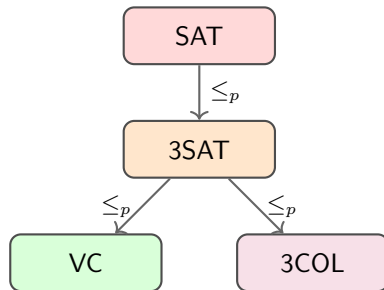
Today's Plan

Four canonical NP-complete problems:

- **SAT** — Boolean satisfiability
- **3SAT** — 3-CNF satisfiability
- **VC** — Vertex Cover (recap)
- **3COL** — Graph 3-Coloring

Three reductions we prove:

- $\text{SAT} \leq_p \text{3SAT}$
- $\text{3SAT} \leq_p \text{VC}$
- $\text{3SAT} \leq_p \text{3COL}$



Boolean Satisfiability (SAT)

Definitions

A **literal** is a Boolean variable x or its negation $\neg x$.

A **clause** is an OR of literals: $(l_1 \vee l_2 \vee \dots \vee l_k)$.

A formula in **conjunctive normal form (CNF)** is an AND of clauses.

SAT (Satisfiability)

Given a CNF formula φ , does there exist a **truth assignment** to the variables that makes φ evaluate to **true**?

Example:

$$\varphi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2) \wedge (\neg x_2 \vee \neg x_3)$$

Assignment $x_1 = T, x_2 = T, x_3 = F$ ✓



SAT Example: A Clause Can Fail Even if the Formula Does Not

Consider

$$\varphi = (x_1 \vee x_2) \wedge (\neg x_1 \vee x_3) \wedge (\neg x_2 \vee \neg x_3).$$

Assignment A: $x_1 = T, x_2 = F, x_3 = T$

- $(T \vee F) = T$
- $(F \vee T) = T$
- $(T \vee F) = T$

So φ is satisfiable under Assignment A. ✓

Assignment B:

$x_1 = F, x_2 = T, x_3 = T$

- $(F \vee T) = T$
- $(T \vee T) = T$
- $(F \vee F) = F$

One false clause makes the whole CNF false. ✗

In CNF, every clause must be true. One satisfied clause helps, but one false clause kills the entire formula.

SAT Example: An Unsatisfiable Formula

Consider

$$\varphi = (x_1 \vee x_2) \wedge (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_2) \wedge (\neg x_1 \vee \neg x_2).$$

Case 1: $x_1 = T$

- Then clauses 3 and 4 become x_2 and $\neg x_2$.
- They cannot both be true at the same time.

Case 2: $x_1 = F$

- Then clauses 1 and 2 become x_2 and $\neg x_2$.
- Again, they cannot both be true.

Checking all assignments:

x_1	x_2	φ
T	T	F
T	F	F
F	T	F
F	F	F

No assignment satisfies all clauses. **X**

An unsatisfiable SAT instance means *every* possible truth assignment fails somewhere.

SAT is NP-Complete

Cook–Levin Theorem (1971/1973)

SAT is NP-complete.

Step 1 — SAT $\in$ NP.

- Certificate: a truth assignment τ (size n).
- Verification: evaluate each clause under τ . Time $O(|\varphi|)$. ✓

Step 2 — SAT is NP-hard.

- Cook (1971) and Levin (1973) independently proved this.
- For every NP problem L and verifier V , the execution of V can be encoded as a polynomial-size SAT formula.
- This gives a poly-time reduction: $L \leq_p \text{SAT}$ for all $L \in \text{NP}$.

SAT is the **starting point** of the NP-completeness universe.



3-SAT

Definition

A CNF formula is in **3-CNF** if every clause has **exactly 3 literals**.

3SAT: Given a 3-CNF formula φ , is it satisfiable?

Example:

$$\varphi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_2 \vee x_3)$$

Assignment $x_1 = T, x_2 = T, x_3 = T$:

- $(T \vee F \vee T) = T$ ✓
- $(F \vee T \vee F) = T$ ✓
- $(F \vee F \vee T) = T$ ✓

Key fact

$3SAT \leq_p SAT$ trivially (3-CNF is CNF). The hard direction is $SAT \leq_p 3SAT$.

3SAT Example: Exactly Three Literal Slots Per Clause

Valid 3-CNF instance:

$$(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_4)$$

Each clause has exactly three literal positions, so this is a 3SAT instance.

Assignment:

$$x_1 = T, x_2 = F, x_3 = F, x_4 = T$$

- $(T \vee T \vee F) = T$
- $(F \vee F \vee T) = T$

Not a 3SAT instance:

$$(x_1 \vee x_2) \wedge (\neg x_2 \vee x_3 \vee x_4 \vee x_5)$$

This is still SAT, but not 3SAT, because one clause has 2 literals and another has 4.

3SAT is not about formulas with *at most* 3 literals per clause here; in this lecture we use the exact-3-literal version.



Reduction: $\text{SAT} \leq_p \text{3SAT}$

Goal: Convert any CNF formula into an *equivalent* 3-CNF formula.

Given clause $C = (l_1 \vee l_2 \vee \dots \vee l_k)$, transform it case by case:

Case	Original clause	Replacement (fresh variables z_i)
$k = 1$	(l_1)	$(l_1 \vee z \vee z') \wedge (l_1 \vee z \vee \neg z') \wedge (l_1 \vee \neg z \vee z') \wedge (l_1 \vee \neg z \vee \neg z')$
$k = 2$	$(l_1 \vee l_2)$	$(l_1 \vee l_2 \vee z) \wedge (l_1 \vee l_2 \vee \neg z)$
$k = 3$	$(l_1 \vee l_2 \vee l_3)$	unchanged
$k > 3$	$(l_1 \vee \dots \vee l_k)$	(see next slide)

Running time: Each clause of length k produces $O(k)$ new 3-clauses and $O(k)$ fresh variables. ✓

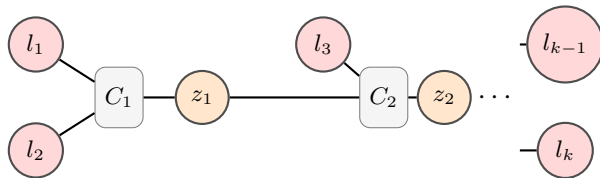


SAT $\leq_p$ 3SAT: The Long-Clause Case ($k > 3$)

For a clause $(l_1 \vee l_2 \vee \dots \vee l_k)$ with $k > 3$:

Introduce fresh variables $z_1, z_2, \dots, z_{k-3}$ and replace with the $k - 2$ clauses:

$$\underbrace{(l_1 \vee l_2 \vee z_1)}_{\text{clause 1}} \wedge \underbrace{(\neg z_1 \vee l_3 \vee z_2)}_{\text{clause 2}} \wedge \dots \wedge \underbrace{(\neg z_{k-4} \vee l_{k-2} \vee z_{k-3})}_{\text{clause } k-3} \wedge \underbrace{(\neg z_{k-3} \vee l_{k-1} \vee l_k)}_{\text{clause } k-2}$$



Correctness: If the original clause is satisfied by some $l_i = T$, set $z_1 = \dots = z_{i-2} = T$ and $z_{i-1} = \dots = z_{k-3} = F$. Every new clause becomes satisfied. Conversely, if the original clause is violated, all $l_i = F$ forces all z_j to satisfy contradictory requirements. ✓



3SAT is NP-Complete

Theorem

3SAT is NP-complete.

Step 1 — 3SAT $\in$ NP. Certificate: truth assignment. Verify each 3-clause in $O(1)$. ✓

Step 2 — 3SAT is NP-hard. We just showed $\text{SAT} \leq_p 3\text{SAT}$. Since SAT is NP-hard:

$$\forall L \in \text{NP} : L \leq_p \text{SAT} \leq_p 3\text{SAT}$$

By transitivity of reductions, $L \leq_p 3\text{SAT}$ for every $L \in \text{NP}$. ✓

How about 2SAT?



Vertex Cover (Recap)

Definition

Given a graph $G = (V, E)$ and integer k .

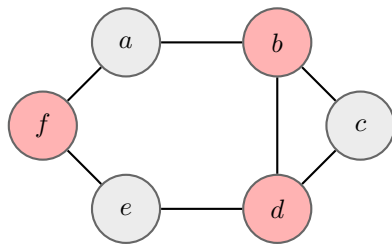
VC: Does there exist a set $S \subseteq V$ with $|S| \leq k$ such that every edge has at least one endpoint in S ?

Example ($k = 3$): ✓Yes

Red vertices $\{b, d, f\}$ cover every edge.

Certificate for VC $\in$ NP:

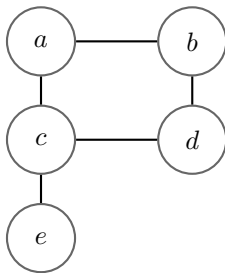
The set S itself. Check each edge has an endpoint in S : $O(|E|)$. ✓



Vertex Cover Example: What Must Be Chosen?

Let G have edges

$$E = \{(a, b), (a, c), (b, d), (c, d), (c, e)\}.$$



Question 1: Is there a vertex cover of size 2?

Yes: choose $S = \{a, d\}$? **X** edge (c, e) is uncovered.

Choose $S = \{b, c\}$? **✓**

- (a, b) covered by b
- (a, c) covered by c
- (b, d) covered by b
- (c, d) covered by c
- (c, e) covered by c



Reduction: $3SAT \leq_p VC$

Input: 3-CNF formula φ with n variables and m clauses.

Construction

Build graph $G = (V, E)$ and set $k = n + 2m$:

Variable gadgets. For each variable x_i , add an edge $(x_i, \bar{x}_i)$.

Clause gadgets. For each clause $C_j = (l_1 \vee l_2 \vee l_3)$, add a triangle $\{u_{j,1}, u_{j,2}, u_{j,3}\}$ with all three edges present.

Cross edges. For each literal l_p in clause C_j at position p , add an edge from $u_{j,p}$ to the corresponding variable node (x_i or $\bar{x}_i$).

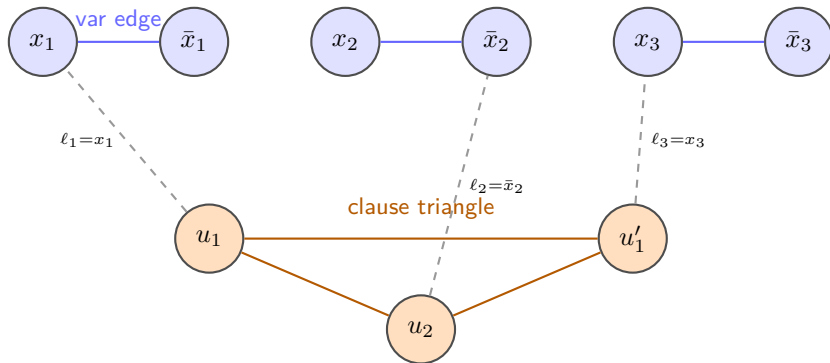
Each variable edge requires ≥ 1 vertex. Each clause triangle requires ≥ 2 vertices to cover its 3 edges.

Any valid VC must use *at least* $n + 2m$ vertices. So the VC of size k must pick *exactly* 1 from each variable edge and *exactly* 2 from each clause triangle.



3SAT $\leq_p$ VC: Example

Formula: $\varphi = (x_1 \vee \neg x_2 \vee x_3)$, $n = 3$, $m = 1$, $k = 3 + 2 = 5$.



Assignment $x_1=T, x_2=F, x_3=T$ satisfies the clause. VC picks $\{x_1, x_2, x_3\}$ (variable edges) and $\{u_2, u'_1\}$ (triangle, omitting u_1 since u_1 's cross edge goes to $x_1 \in S$).



3SAT $\leq_p$ VC: Correctness

Claim

φ is satisfiable $\iff G$ has a vertex cover of size $k = n + 2m$.

$(\Rightarrow)$ SAT assignment $\Rightarrow$ VC.

- 1 For each variable x_i : add the literal node that is *true* (either x_i or $\bar{x}_i$) to S . This covers all variable edges. (Picks n vertices.)
- 2 For each clause C_j : since the clause is satisfied, some literal node $u_{j,p}$ has a cross edge leading to a node *already in* S . Add the other two triangle nodes to S . This covers all triangle edges and cross edges. (Picks 2 vertices per clause.)

Total: $|S| = n + 2m = k$. ✓

$(\Leftarrow)$ VC of size $k \Rightarrow$ SAT assignment.

- Exactly 1 node chosen from each variable edge $\rightarrow$ defines a truth assignment.
- Exactly 2 nodes chosen from each clause triangle $\rightarrow$ the unchosen node $u_{j,p}$ has its cross edge to a literal node in $S \rightarrow$ that literal is true $\rightarrow$ clause is satisfied. ✓



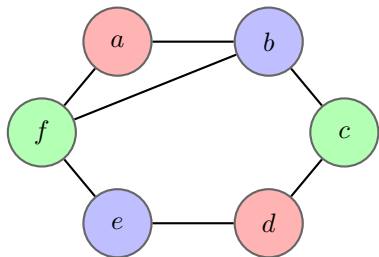
Graph 3-Coloring (3COL)

Definition

Given an undirected graph $G = (V, E)$.

3COL: Does there exist a function $c : V \rightarrow \{1, 2, 3\}$ such that $c(u) \neq c(v)$ for every edge $(u, v) \in E$?

Example — 3-colorable:

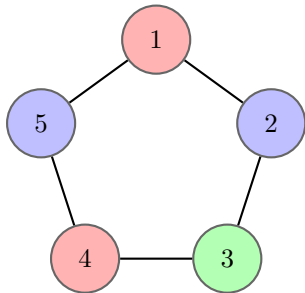


Certificate for 3COL $\in$ NP:

- Certificate: the coloring c .
- Verify each edge (u, v) : check $c(u) \neq c(v)$. Time $O(|E|)$. ✓

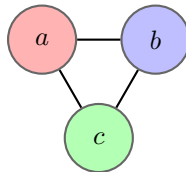
3COL Example: Adjacent Vertices Matter, Not All Vertices

A 3-coloring of a 5-cycle:



Vertices 1 and 4 both use red, which is allowed because they are not adjacent.

Why a triangle already needs 3 colors:



Every pair is adjacent, so no color can be reused inside the triangle.

Reduction: $3SAT \leq_p 3COL$ — Overview

Input: 3-CNF formula φ with variables $x_1, \dots, x_n$ and clauses $C_1, \dots, C_m$.

Construction

Build graph G using three types of gadgets. Colors used: **T** (true), **F** (false), **B** (base).

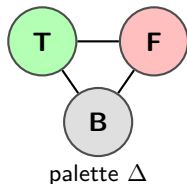
- 1 **Palette triangle.** Three special nodes T, F, B, all pairwise adjacent. Forces the coloring to distinguish exactly three roles.
- 2 **Variable gadgets.** One per variable x_i : triangle $\{x_i, \bar{x}_i, B\}$. Forces exactly one of $x_i, \bar{x}_i$ to get color T and the other F.
- 3 **Clause gadgets.** One per clause $(l_1 \vee l_2 \vee l_3)$: a 6-node two-triangle (OR gadget) attached to l_1, l_2, l_3 and the palette F node. Is 3-colorable $\iff$ at least one literal is colored T.

φ is satisfiable $\iff G$ is 3-colorable.

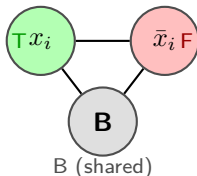


$3SAT \leq_p 3COL$: Palette and Variable Gadgets

Palette triangle (always present):



Variable gadget for x_i :



Why the variable gadget works:

- B is already colored B (fixed by palette Δ).
- x_i and $\bar{x}_i$ both connect to B: neither can be B.
- They connect to each other: must differ.
- One gets T, the other F: models the truth value of x_i .

Invariant after variable gadgets

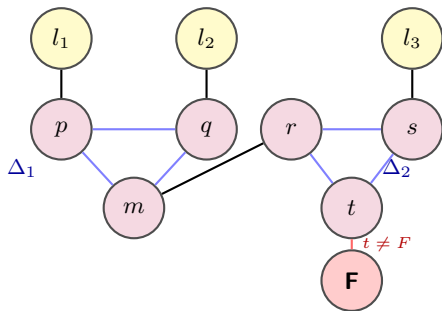
Under any valid 3-coloring:

$$\text{color}(x_i) \in \{T, F\}, \quad \text{color}(x_i) \neq \text{color}(\bar{x}_i).$$

$$x_i \text{ is true} \iff \text{color}(x_i) = T.$$

$3SAT \leq_p 3COL$: Clause Gadget

For clause $C = (l_1 \vee l_2 \vee l_3)$, attach a **two-triangle OR gadget** with 6 new nodes p, q, m, r, s, t :



Edges added per clause:

- Δ_1 : $p-q-m-p$
- Cross: l_1-p, l_2-q
- Bridge: $m-r$
- Δ_2 : $r-s-t-r$
- Cross: l_3-s
- $t-F$ (forces $t \neq F$)

Key property

3-colorable $\iff$ at least one l_i colored T.

All $l_i=F$: forces $m=F$,
then forces $t=F$ — contradicts
 $t-F$ edge. ✗

$3SAT \leq_p 3COL$: Correctness

Theorem

φ is satisfiable $\iff G$ is 3-colorable.

($\Rightarrow$) Satisfying assignment $\Rightarrow$ 3-coloring.

- 1 Color palette: $T \rightarrow T, F \rightarrow F, B \rightarrow B$.
- 2 Color x_i with T (resp. F); $\bar{x}_i$ gets the opposite. Variable triangles proper. ✓
- 3 Clause satisfied $\Rightarrow$ some l_i colored $T \Rightarrow$ clause gadget 3-colorable. ✓

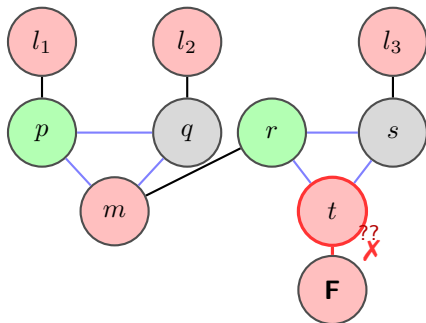
($\Leftarrow$) 3-coloring $\Rightarrow$ satisfying assignment.

- 1 Palette forces roles T, F, B .
- 2 Variable gadgets: $\tau(x_i) = \text{true} \iff \text{color}(x_i) = T$. ✓
- 3 Clause gadget properly colored $\Rightarrow$ some l_i colored $T \Rightarrow$ clause satisfied. ✓

Running time: $|V|, |E| = O(n + m)$; construction $O(n + m)$. ✓



Clause Gadget: All Literals $F \Rightarrow$ Contradiction



Legend:

● T ■ F ■ B

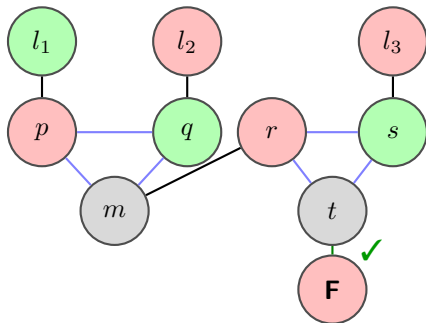
Forcing chain:

- 1 $l_1=l_2=F$: $p, q \notin \{F\}$ and $p \neq q$, so $\{p, q\} = \{T, B\}$.
- 2 Δ_1 needs all 3 colors $\Rightarrow m=F$.
- 3 $m=F$: $r \notin \{F\}$; $l_3=F$: $s \notin \{F\}$.
- 4 $r \neq s$, both in $\{T, B\} \Rightarrow t=F$.
- 5 $t-F$ edge: $t \neq F$. ✗

No valid 3-coloring exists when all literals are false.



Clause Gadget: $l_1=T, l_2=l_3=F \Rightarrow$ Valid 3-Coloring



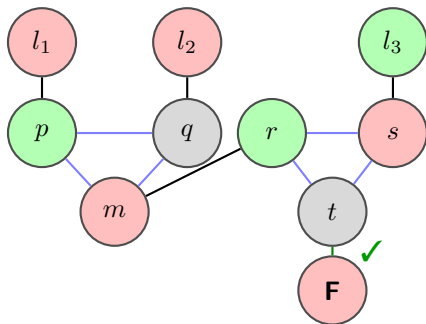
Legend:

● ■ T ■ F ■ B

Why it works:

- $l_1=T$: $p \neq T$, choose $p=F$.
- $l_2=F$: $q \neq F$, choose $q=T$.
- $\Delta_1=\{F, T, B\}$: $m=B$.
- $m=B$: $r \neq B$, choose $r=F$.
- $l_3=F$: $s \neq F$, choose $s=T$.
- $\Delta_2=\{F, T, B\}$: $t=B$.
- $t=B \neq F$: ✓

Clause Gadget: $l_3=T, l_1=l_2=F \Rightarrow$ Valid 3-Coloring



Legend:

● ■ T ■ F ■ B

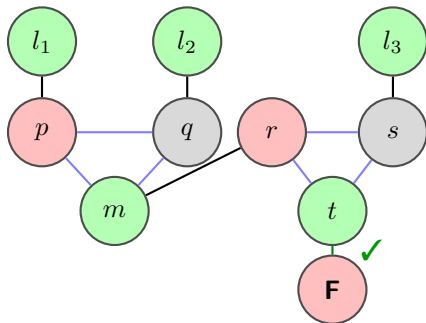
Why it works:

- $l_1=l_2=F$: $p, q \notin \{F\}$, $p \neq q$.
- Choose $p=T$, $q=B \Rightarrow m=F$.
- $m=F$: $r \neq F$, choose $r=T$.
- $l_3=T$: $s \neq T$, choose $s=F$.
- $\Delta_2=\{T, F, B\}$: $t=B$.
- $t=B \neq F$: ✓

Even with $l_1=l_2=F, l_3=T$ at the second triangle gives enough freedom.



Clause Gadget: All Literals $T \Rightarrow$ Valid 3-Coloring



Legend:

● ■ T ■ F ■ B

Why it works:

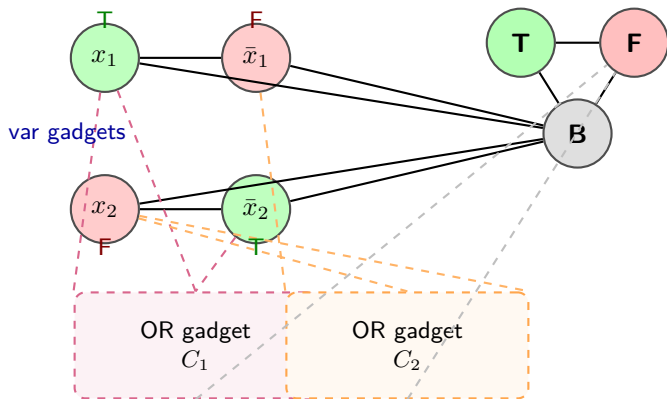
- $l_1=l_2=T$: $p, q \notin \{T\}$, $p \neq q$.
- Choose $p=F$, $q=B \Rightarrow m=T$.
- $m=T$: $r \neq T$, choose $r=F$.
- $l_3=T$: $s \neq T$, choose $s=B$.
- $\Delta_2=\{F, B, T\}$: $t=T$.
- $t=T \neq F$: ✓

Summary

The gadget is 3-colorable $\iff$ at

$3SAT \leq_p 3COL$: Full Example

Formula: $\varphi = (x_1 \vee \neg x_2 \vee x_1) \wedge (\neg x_1 \vee x_2 \vee x_2)$ (2 vars, 2 clauses)



Assignment $x_1=T, x_2=F$: colored above literal nodes. Both clause gadgets have a T literal and can be properly 3-colored.

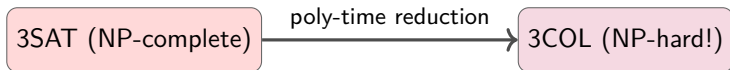
3COL is NP-Complete

Theorem

3COL is NP-complete.

Step 1 — 3COL $\in$ NP. Certificate: the coloring c . Check each edge: $O(|E|)$. ✓

Step 2 — 3COL is NP-hard. We showed $3SAT \leq_p 3COL$:



Every $L \in \text{NP}$ reduces to $3SAT \leq_p 3COL$ by transitivity. ✓

Remark: k -Coloring

- **2-COL** (bipartiteness testing): solvable in $O(|V| + |E|)$ by BFS/DFS.
- **3-COL**: NP-complete (just proved).
- **k -COL for $k \geq 3$** : NP-complete (reduce 3COL to k COL via K_k attachment).